

114 學年第 1 學期

輔仁大學
人工智慧與資訊安全學士學位學程
專題報告

題目：

後量子密碼遷移之加密演算
法自動化盤點框架研究

學號 412580266 名字—李昺園

學號 412580333 名字—黃馨賢

學號 412580345 名字—余采芸

指導老師：勤業眾信 陳威棋 副總

114 年 12 月

目錄

第一章、擬定專題製作計畫

第一節、目的說明	1
第二節、專題簡介	1
第三節、專案管理規劃	3
第四節、可行性分析	4

第二章、系統需求分析

第一節、目的說明	5
第二節、需求分析方法與工具	5
第三節、問題陳述	5
第四節、功能性需求	6
第五節、非功能性需求	7

第三章、系統設計

第一節、目的說明	9
第二節、設計方法與工具	9
第三節、軟體架構之準則	10
第四節、系統元件與模組設計規劃	10
第五節、人機介面設計	10
第六節、檔案/資料庫設計	11

第七節、執行設計追溯	12
第四章、系統實作	
第一節、目的說明	14
第二節、系統實作說明	14
第三節、程式原始碼	16
第五章、系統測試	
第一節、目的說明	44
第二節、測試環境	44
第三節、測試計畫	44
第六章、使用手冊	
第一節、目的說明	48
第二節、系統需求	48
第三節、安裝程序	49
第四節、操作程序	49
第五節、維護程序	50
第七章、進度報告	
第一節、目的說明	52
第二節、進度表	52
第八章、專題結論	

第一節、專題成果與心得	53
第二節、未來展望	53
第三節、銘謝	54
參考文件	55

第一章、擬定專題製作計畫

第一節、目的說明

本章節旨在定義本專題的執行範圍、時間表、資源分配、管理方法與預期成果。

1. 專案目標：開發一個高效、精準、跨語言的靜態分析工具，用於自動盤點程式碼中的量子脆弱性密碼學資產。
2. 文件用途：作為專題團隊與指導教授之間進度和品質控制的依據。

第二節、專題簡介

一、背景介紹

1. 潛在的量子威脅：

隨著量子計算技術快速發展，傳統基於 RSA、ECC 的公鑰密碼體系將在量子電腦成熟後失效。這種威脅已不再是理論，是全球 IT 基礎設施必須面對的嚴峻挑戰。

2. 企業現況挑戰：

軟體架構龐大且複雜，RSA 與 ECC 廣泛存在於程式、協定、套件與憑證中，造成盤點難度高，人工盤點密碼學資產既耗時又易遺漏，因此，企業需要一個自動化框架，可以有效支援複雜的後量子密碼遷移。

二、問題陳述

1. 軟體資產盤點的規模與語義挑戰：

缺乏語義透明度，RSA 與 ECC 等量子脆弱資產的追溯，要求的不僅是字串比對，更需要理解程式碼的語境。

2. 現有自動化工具的技術局限

- 傳統工具依賴字串比對：傳統的掃描工具大多只能在文字層面進行模式匹配，無法區分程式碼、註解或字串常數。
- 誤報與漏報問題嚴重：這種技術限制會導致高誤報率，或因無法解析複雜的語義而造成高漏報率。
- 無法進行深度分析：傳統方法無法解析函式參數的語義。

三、研究目的

本研究的目的是提出跨語言自動化盤點框架，以應對企業需求。框架整合了規則庫，結合 CBOM 流程及 HTML 網頁報告。

1. 建構高精準度的密碼資產識別機制：

利用抽象語法樹（AST）靜態分析技術，建構一套高精準度、低誤報率的識別機制。

目標是解決傳統正規表達式（Regex）工具容易產生高誤報或高漏報的問題，精準判斷密碼學的使用語境。

2. 實現跨語言盤點與資產追溯：

開發一套跨語言的解析器核心，實現對多語言程式碼的統一盤點與追溯，我們目前著重在 Python 和 Java 的部分。

3. 建立標準化的密碼學物料清單（CBOM）報告生成器，將盤點結果結構化，以提升軟體供應鏈在 PQC 遷移中的數據透明度。

4. 將掃描數據即時轉換為可互動的圓餅圖，並嵌入 HTML 網頁報告中，提升報告的分析深度和使用者體驗。

5. 設計一套可支援 CLI 呼叫的自動化框架架構

四、傳統方法與本研究的比較

面向	傳統方法	本研究工具
盤點效率	依賴人工代碼審查或簡單的字串比對，效率低且誤報率高。	採用 AST 靜態分析，實現從專案級掃描到報告輸出的全自動化流程。
精確度	難以判斷密碼學函式調用的參數配置是否安全。	實作 AST 遍歷和規則引擎，精確分析函式參數和上下文，實現語義級識別。
報告標準	輸出格式缺乏一致性，難以被其他工具使用。	輸出標準化的 CBOM JSON 檔案，並提供 Plotly HTML 互動式報告。

五、主要功能

1. 專案掃描：使用 os.walk 實現對多層級專案目錄的遞迴掃描。
2. 跨語言解析：支援 Python（內建 ast）和 Java（javalang）的 AST 轉換。
3. 規則匹配：集中式 PQC 知識庫驅動的規則引擎，執行精準的 AST 節點匹配。
4. 報告輸出：輸出標準化 CBOM JSON（FR-05）和視覺化 HTML 網頁報告（FR-06）。

第三節、專案管理規劃

一、工作分配（分工表）

李昺園（核心引擎）：

1. 開發 Python CLI
2. 開發 Python AST 引擎、Java AST 引擎（解析器）
3. 系統測試

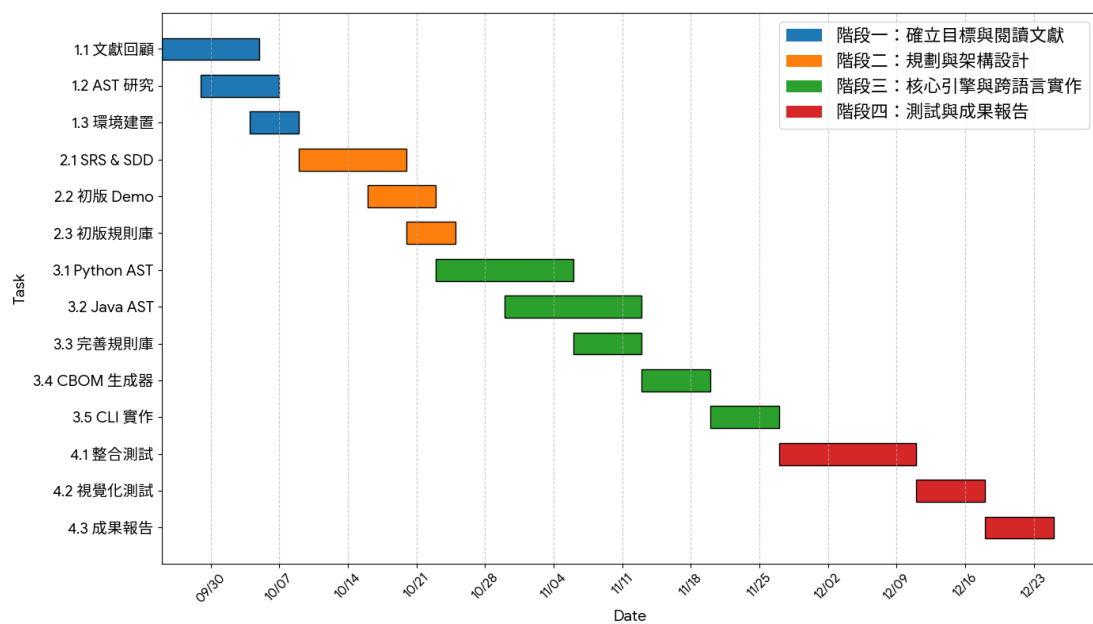
余采芸（輸出與報告）：

1. 開發 CBOM 生成器及 HTML 網頁報告
2. 製作書面報告
3. 系統測試

黃馨賢（規則庫）：

1. 分析及新增規則
2. 系統測試

二、進度規劃（甘特圖）



第四節、可行性分析

一、技術可行性

項目	是否可行	分析
AST 解析	是	Python 的內建 ast 模組成熟，而 javalang 和 pycparser 等開源庫已提供 Java 和 C/C++ 的 AST 轉換能力。
規則引擎分離	是	採用 Visitor 模式搭配獨立的 RuleEngine，實現高內聚、低耦合的模組化設計。
Plotly 輸出	是	Plotly 函式庫能直接生成獨立的 HTML 檔案，無需額外的伺服器架設。

總結：技術上完全可行。所需技術成熟且開源函式庫支持度高，主要挑戰在於規則庫的精準度與 AST 遍歷的複雜性。

二、經濟可行性

1. 本專題主要使用開源軟體與函式庫 (Python, ast, javalang, pandas, plotly)，無需額外的授權費用。
2. 主要成本為人力時間投入。經濟效益在於開發完成後，能以零成本提供企業或開發者進行大規模的 PQC 風險盤點。

三、操作可行性

1. 操作簡便：採用 CLI 介面 (FR-07)，操作流程簡單，只需輸入目標路徑即可運行。
2. 輸出標準：輸出的 CBOM JSON 報告 (FR-05) 便於未來與企業現有的漏洞管理系統或 CI/CD 流程自動集成，實用性高。

總結：系統設計符合自動化和標準化趨勢，操作上對使用者和未來系統集成都高度可行。

第二章、系統需求分析

第一節、目的說明

本章節旨在清晰、完整地定義本專題的功能和非功能需求，作為專案開發、驗證和最終交付的官方標準。

目標：

1. **規範開發範圍：** 作為開發團隊的藍圖，詳細說明系統必須實現的所有功能（例如：跨語言 AST 解析、規則引擎運行、雙模報告輸出）和性能指標。
2. **指導驗證與測試：** 為測試階段提供明確的驗收標準。確保最終交付的系統能夠準確地識別程式碼中的量子脆弱性，滿足專題目標。
3. **溝通基礎：** 作為開發者、專題指導教授和潛在使用者之間溝通的共同基礎，確保各方對專案的目標、能力和限制有統一的理解。

第二節、需求分析方法與工具

1. 文獻探討： 深入研究 NIST PQC 遷移指南、OWASP 加密弱點（如 CWE-327）、以及現有 AST 分析工具（如 Bandit, pycparser）的設計原理及其規則庫。
2. 原型法： 將為每種語言（Python, Java, C++）開發小型的 AST 遍歷器原型，以驗證偵測特定加密模式（如 RSA 密鑰長度、AES-ECB 模式）的技術可行性。

第三節、問題陳述

當前困境：

組織在 PQC 遷移前，急需盤點程式碼中的加密資產。然而，現有的靜態分析工具多基於「模式匹配」（如 RegEx 或 Sengrep 的 pattern），這導致了兩個主要問題：

1. 高誤報率： 它們很難區分註解中的 RSA 和實際執行的 `RSA.generate()`。
2. 缺乏深度上下文： 它們無法輕易檢查 `RSA.generate(1024)` 中的參數值（密鑰長度），或 `AES.new()` 中的關鍵字參數（如 `mode=AES.MODE_ECB`）。

目標：

直接在 AST 層面進行分析。只有遍歷抽象語法樹，才能精確地識別函式呼叫、檢查傳入的參數值、並追蹤變數，從而實現對 PQC 相關風險（如密鑰過短、使用不安全模式、缺少 IV）的高精度偵測。

第四節、功能性需求 (FR)

編號	項目	說明
FR-01	專案級掃描	接受一個目錄路徑，並遞迴掃描其中所有支援的原始碼檔案
FR-02	跨語言 AST 解析器	系統必須具備針對 Python, Java, C/C++ 原始碼的 AST 解析能力
FR-03	集中式 PQC 知識庫	建立一個集中的、易於擴充的「知識庫」(例如 Python 字典或 YAML 檔案)，用以定義所有 PQC 規則、風險和修復建議
FR-04	AST 遍歷與規則匹配引擎	系統應具備核心的「訪問者」(Visitor) 模組，用於遍歷 AST 節點，並將節點資訊與 FR-03 知識庫進行比對
FR-05	標準化 CBOM 報告	將所有分析結果匯總，並生成一份符合 CycloneDX 標準的 CBOM (JSON 格式) 報告
FR-06	HTML 網頁報告	將 Plotly 生成的圖表 HTML 內容，寫入一個單一的 PQC_Report.html 檔案中，完成報告的可視化輸出
FR-07	命令行介面 (CLI)	提供一個 CLI 介面，以供使用者與 CI/CD 流程自動化呼叫

一、專案級掃描 (FR-01)：

1. 使用 `os.walk` 實現遞迴掃描
2. 建立一個檔案分派器 (dispatcher)，根據副檔名 (`.py`, `.java`, `.c`, `.cpp`) 呼叫相應的 AST 解析函數

二、跨語言 AST 解析器 (FR-02)：

1. Python：使用內建的 `ast` 模組來解析 Python 程式碼 (`ast.parse`)
2. Java：引入 `javalang` 函式庫來解析 Java 程式碼 (`javalang.parse.parse`)
3. C/C++：引入 `pycpparser` 函式庫

三、集中式 PQC 知識庫 (FR-03)：

1. 從現有工具的規則庫中，分析並篩選符合 PQC 遷移的規則

2. 每條規則應至少包含 RuleID, Type, Message, FixSuggestion, PQC-Status (vulnerable, safe, n/a), Algorithm (RSA, AES...)

四、AST 遍歷與規則匹配引擎 (FR-04)：

1. Python：實作一個繼承自 ast.NodeVisitor 的類別
2. Java：遍歷 javalang.tree 節點
3. C/C++：實作一個繼承自 c_ast.NodeVisitor 的類別
4. 這些 Visitor 類別不應包含硬編碼的 if/else 規則。它們應只負責「收集」AST 節點資訊，然後將此資訊傳遞給一個獨立的 RuleEngine 模組，由該引擎去比對 (FR-03) 知識庫。

五、標準化 CBOM 報告 (FR-05)：

1. 建立一個「合約」介面，用於接收 (FR-04) 引擎產出的標準化結果
2. 使用 cyclonedx-python-lib 函式庫
3. 將每一個 PqcFinding 物件轉換為一個 CycloneDX Component，並利用 properties 欄位儲存 PQC 相關的元數據

六、HTML 網頁報告 (FR-06)：

1. 數據準備：程式利用 pandas 對原始 findings 列表進行分組和計數，為 Plotly 繪圖準備數據結構。
2. 圖表生成：呼叫 plotly.graph_objects 相關方法，生成互動式圖表。
3. HTML 輸出：將 Plotly 生成的圖表 HTML 內容，寫入一個單一的 PQC_Report.html 檔案中，完成報告的可視化輸出。

七、命令行介面 (FR-07)：

1. 使用 argparse 或 click 函式庫

至少支援 `pqc-scanner --path <project-dir> --output <bom-file.json>` 格式

第五節、非功能性需求 (NFR)

類別	編號	描述	驗收標準
性能	NFR-P1	掃描效率	程式碼分析速度不低於 500 行/秒。
	NFR-P2	資源消耗	掃描 10 萬行程式碼時，記憶體使用峰值不超過 1 GB。
	NFR-P3	報告生成延遲	掃描完成後，報告生成總時間不超過 5 秒。
安全性	NFR-S1	本地運行	核心掃描、解析和規則匹配時，不

			依賴外部網路連接。
	NFR-S2	文件操作隔離	系統應不嘗試修改或刪除原始碼，僅讀取和寫入報告檔案。
	NFR-S3	數據敏感性	輸出報告中不得包含原始碼片段，僅輸出發現的位置和分類信息。
可用性	NFR-U1	部署簡便性	從 Git Clone 到 CLI 成功運行，所需命令列步驟不應超過 5 步。
	NFR-U2	錯誤訊息清晰度	輸出錯誤訊息時，需提供具體、清晰的原因提示。
	NFR-U3	報告可讀性	Plotly 圖表必須具備懸停顯示數據的功能，且風險等級顏色區分清晰。
可擴展性	NFR-M1	模組化設計	規則引擎模組與 AST 遍歷器之間低耦合。
	NFR-M2	知識庫更新	添加一條新的 PQC 規則，僅需修改 PQC 知識庫檔案即可。
	NFR-M3	未來語言擴展	存在抽象的 Parser 介面，便於未來添加新的語言解析器。

第三章、系統設計

第一節、目的說明

本章節旨在詳細說明「跨語言 PQC 靜態 AST 掃描器」的系統架構、模組設計、介面以及資料結構。本文件是 SRS 的技術實作藍圖，其內容將指導開發團隊如何依據 SRS 需求，建構一個高精度、高內聚、低耦合的 PQC 分析引擎與服務。

目標：

- 1. **精準盤點：** 利用抽象語法樹（AST）實現跨語言（Python/Java）的語義級密碼資產識別。
- 2. **標準輸出：** 輸出符合行業標準（CBOM JSON）的數據報告，便於與其他安全工具鏈集成。
- 3. **直觀呈現：** 透過 Plotly 視覺化技術，提供使用者友善的 HTML 網頁報告，加速風險決策。

第二節、設計方法與工具

一、設計方法論：模組化與事件驅動

- 1. 採用模組化設計原則，將系統功能劃分為獨立的元件（例如：Scanner、Parser、RuleEngine 和 Reporter），以提高代碼的可維護性、可測試性和未來擴展性。
- 2. 核心流程遵循數據流和事件驅動的概念：文件遍歷（事件）觸發解析器，解析結果（數據）匯入規則引擎。

二、核心設計工具與函式庫

類別	工具/函式庫	功能
程式語言	Python 3.9 以上	作為框架的主要開發語言，利用其強大的文件操作和生態系統。
靜態解析	javalang, pycparser, Python ast	實現跨語言的 AST 生成與遍歷。
數據處理	pandas	用於掃描結果的結構化、清洗與聚合，為視覺化報告提供統計數據。
數據視覺化	plotly	負責生成互動式的 HTML

		圖表，作為風險評估的直觀呈現。
--	--	-----------------

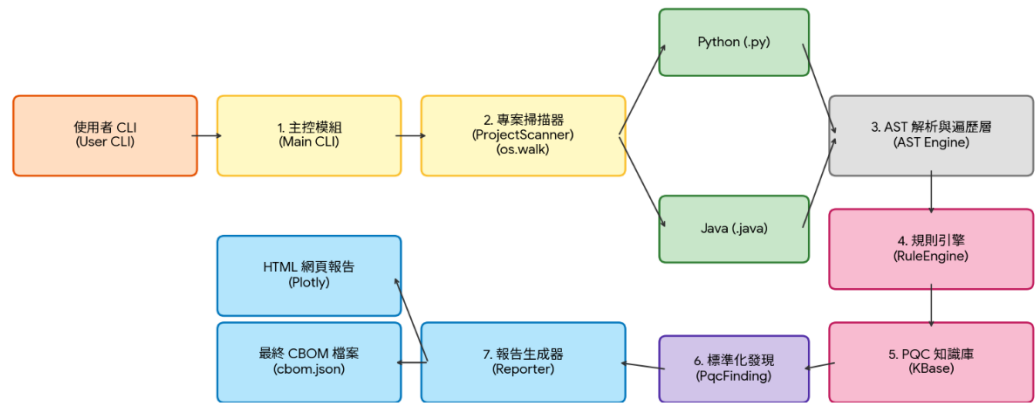
第三節、軟體架構之準則

本系統採用的主要軟體架構準則包括：

3. 單一職責原則：每個模組僅負責一項核心功能。例如，RuleEngine 僅負責匹配規則，而 Reporter 僅負責格式化輸出。
4. 關注點分離：將核心業務邏輯（AST 遍歷與規則匹配）與 I/O 操作（檔案讀取、報告寫入）完全分離，確保核心邏輯不受 I/O 變化影響。
5. 命令列介面（CLI）模式：採用精簡的 CLI 驅動設計，有利於系統在不同操作系統環境下快速部署和運行。

第四節、系統元件與模組設計規劃

系統架構流程圖：



系統的核心處理流程圍繞著五個主要模組構成，它們協同工作以完成從程式碼讀取到報告輸出的整個流程。

模組名稱	程式碼對應	核心功能
主控模組	main 函數 / CLI 處理	接收 CLI 參數，初始化環境，並協調調度其他核心模組的運行順序。
專案掃描器	ProjectScanner	遞迴遍歷目標目錄，根據檔案類型（如 .py 或 .java）將檔案分派給對應的解析器。
語言解析器	Parser 介面 / javalang / ast	將原始碼轉換為 AST 結構。

規則引擎	RuleEngine / PQC 知識庫	遍歷 AST，根據內建規則（如弱密鑰長度、不安全模式）匹配密碼資產，生成標準化的 PqcFinding 物件。
報告生成器	Reporter / pandas / plotly	接收所有 PqcFinding 物件，生成標準化 CBOM JSON 和 Plotly HTML 網頁報告。

第五節、人機介面設計

一、輸入介面：命令列介面（CLI）

1. 設計原則：簡潔、高效、參數化。
2. 用途：專注於自動化部署和批次處理。使用者僅需透過參數傳入目標路徑即可運行。
3. 指令範例：python pqc_ast_scanner.py [目標目錄或檔案路徑]

二、輸出介面：視覺化網頁報告

3. 設計原則：直觀、互動、易於分析。
4. 用途：透過 Plotly 生成的 PQC_Report.html 提供圖表形式的風險分佈，輔助風險決策者快速掌握專案的量子脆弱性現狀。

第六節、檔案/資料庫設計

本專題的設計採用無狀態（Stateless）運行，主要依賴於文件 I/O 而非傳統資料庫。

一、輸入檔案設計

1. 原始碼檔案：專案支援讀取和解析常見的原始碼檔案（.py, .java）。
2. PQC 知識庫（KBase）：規則資料以硬編碼或簡單的配置檔案形式存儲在程式內部，用於規則引擎的匹配。

二、輸出檔案設計

檔案名稱	格式	用途	結構化依據
PQC_CBOM_Inventory.json	JSON	機器可讀的密碼資產清單，用於安全工具鏈的集成。	簡化版 CycloneDX/SPDX 標準
PQC_Report.html	HTML	人機友善的視覺化報告，內嵌 Plotly 圖表。	視覺化統計數據

三、資料結構設計 (PqcFinding 物件)

所有掃描發現都會被標準化為 PqcFinding 數據物件，確保數據在系統內部的一致性。

屬性名稱	數據類型	說明
file_path	String	發現弱點的原始碼檔案路徑
line_number	Integer	弱點所在的行數
asset_type	String	密碼資產類型 (e.g., RSA, AES, ECC)
status	String	量子脆弱性狀態 (e.g., VULNERABLE (QUANTUM), WEAK)
recommendation	String	基於 KBase 的遷移建議

第七節、執行設計追溯

本追溯表格證明所有功能需求 (FR, Functional Requirements) 均已被系統元件和模組設計所支持。

SRS 需求 ID	核心功能說明	系統元件/模組	實作細節與追溯方式	驗證標準
FR-01	專案級掃描	ProjectScanner 類別 (含 os.walk), Dispatcher 邏輯	使用 Python 內建的 os.walk 實現高效遞迴遍歷。Dispatcher 檢查檔案副檔名 (.py, .java 等)，並將路徑傳遞給對應的解析器。	成功掃描一個包含多層級、多語言檔案的專案，且無遺漏或重複掃描。
FR-02	跨語言 AST 解析器	Python ast, javalang.parse, pycparser	各語言解析器只負責將原始碼轉換為其語言的標準 AST 結構 (ast.parse, javalang.parse.parse)，不執行任何規則判斷。	成功解析包含複雜結構的 Python/Java 測試檔案，且 AST 節點數量與預期相符。
FR-03	集中式 PQC 知識庫	PqcRule 類別, Rule 數據字典	知識庫 (KBase) 以結構化數據集中存儲，確保每條規則包含 RuleID 和 PQC-Status 等關鍵元數據。	知識庫中的所有規則均符合定義的結構，且能被 RuleEngine 成功加載。
FR-04	AST 遍歷	PythonVisitor, JavaVisitor,	Visitor 類別僅繼承 NodeVisitor 實作節點收	遍歷結果正確傳遞給

	與規則匹配引擎	RuleEngine 模組	集，將節點資訊傳給獨立的 RuleEngine。 RuleEngine 模組負責執行 FR-03 的知識庫比對，實現邏輯與遍歷的分離。	RuleEngine，且 RuleEngine 能依據 KBase 準確生成 PqcFinding 物件。
FR-05	標準化 CBOM 報告	cyclonedx-python-lib, CbomReporter	使用 cyclonedx-python-lib 函式庫，將標準化的 PqcFinding 物件，轉譯為符合 CycloneDX 規範的 Component。PQC 相關資訊儲存於 properties 欄位。	輸出的 JSON 檔案結構符合 CycloneDX 標準，且包含所有 PQC 相關元數據。
FR-06	HTML 網頁報告	pandas, plotly 函式庫, HtmlReporter	利用 pandas 對 findings 進行統計聚合，使用 plotly.graph_objects 生成可互動圖表，並將 HTML 圖表內容直接寫入單一的 PQC_Report.html。	生成的 HTML 檔案能在瀏覽器中開啟，Plotly 圖表可互動，且統計數據與原始發現數量一致。
FR-07	命令行介面 (CLI)	argparse 或 click 函式庫	實作 CLI 介面，至少支持 --path <project-dir> (目標路徑) 和 --output <bom-file.json> (輸出路徑) 等核心參數，確保系統易於運行和自動化。	系統可通過單一 CLI 指令運行，且參數解析正確，能成功觸發掃描流程。

第四章、系統實作

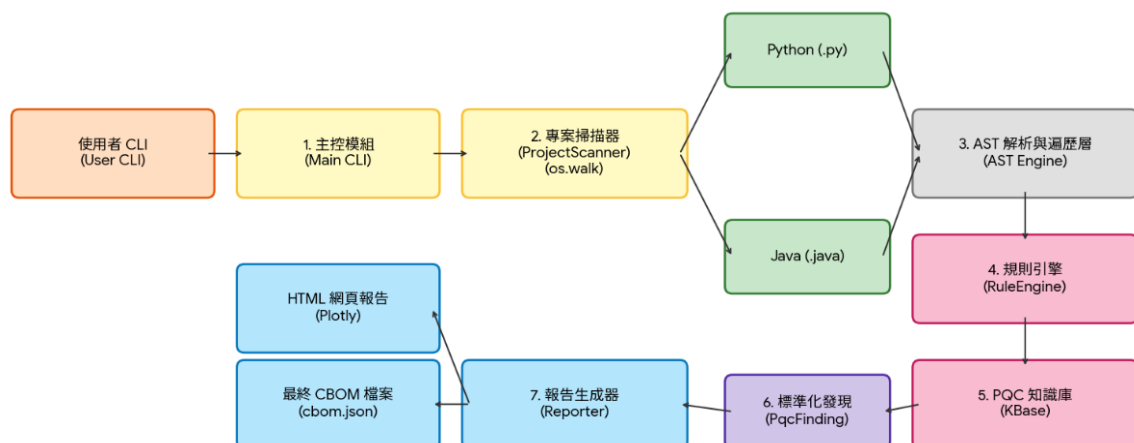
第一節、目的說明

本章節旨在摘要描述本專題的實作過程、技術細節、程式碼撰寫準則，以及最終的實作成果。

1. 文件目的：提供一個系統實作規格，作為指導後續程式碼維護、功能擴充和軟體元件重用的官方文件。
2. 內容描述：本章內容涵蓋了系統的程式語言與版本、實作版本狀態、所使用的相關軟硬體資源，並說明了程式撰寫、註解及核心模組定義的過程。

第二節、系統實作說明

一、系統架構圖



二、程式語言與版本

項目	描述
主要開發語言	Python 3.9+（基於對 AST 處理和數據科學庫的良好支持）
支援解析語言	Python, Java（透過相關函式庫支援）

三、實作版本

項目	描述
目前實作版本	v1.0.0
版本狀態	已完成所有核心功能（FR-01 至 FR-07）的集成與初步測試，進入性能優化階段。

四、相關軟體及函式庫

本系統的實作依賴以下第三方函式庫來實現跨語言解析、數據處理與視覺化：

類別	函式庫/軟體	用途說明
Java 解析器	javalang	將 Java 原始碼轉換為 AST 節點結構。
C/C++ 解析器	pycpparser	支援 C/C++ 原始碼的 AST 轉換。
數據處理	pandas	用於掃描結果的分組、聚合，以計算風險統計數據。
報告生成	plotly	生成互動式的 HTML 報告圖表。
CBOM 標準化	cyclonedx-python-lib	將 PqcFinding 數據轉換為標準的 CycloneDX JSON 格式。
CLI 介面	argparse 或 click	實作命令列介面 (CLI) 的參數解析和指令結構。

五、程式撰寫準則與核心模組定義

本專題在程式撰寫階段遵循以下準則，以確保程式碼的可讀性和可維護性：

1. 程式註解與文件化

- 註解規範：所有公開的類別 (Class)、方法 (Method) 和函式 (Function) 均採用 Python Docstring 規範進行詳細註解，說明其功能、參數 (Params) 和回傳值 (Returns)。
- 目的：確保其他開發者能夠迅速理解該程式單元的職責和使用方式。

六、系統技術與功能說明

項目	程式碼對應函數/類別	功能
CLI 介面	if __name__ == __main__	程式的 CLI 入口點，負責接收使用者傳入的檔案或目錄路徑
專案自動化遞迴掃描	scan_project_recursive	依據路徑類型，決定掃描單一檔案或遞迴遍歷整個目錄
知識庫 (KBase)	PQC_KNOWLEDGE_BASE 字典	儲存弱雜湊 (MD5/SHA1)、弱加密 (DES)、量子脆弱資產 (RSA/ECC) 等規則
Python AST 引擎	scan_python, PQC_AST_Visitor	● 基於 ast.NodeVisitor 實現精準的語義分析 ● 能檢查 RSA 密鑰長度、AES/ECB 模式、和 IV 缺失等複雜的使用樣式風險
Java AST 引	scan_java	● 使用 javalang 函式庫解析

引擎		Java 程式碼 ● 能偵測 getInstance 中的 MD5/SHA1、DES 和 AES/ECB 模式
標準化發現	report_finding 函數	負責將 AST 節點轉換為統一的 Python 字典格式 (RuleID, Type, Location 等)
CBOM 報告	generate_cbom 函數	● 彙整所有發現，並生成 CBOM JSON 檔 ● 自動為每個資產判斷 pqc_status
HTML 網頁報告	圖表視覺化：plotly 函式庫 數據預處理：pandas 函式庫 文件生成： generate_full_report_html() 輸出結果：PQC_Report.html"	● 將掃描結果即時轉換為圖表 ● 對原始的 PqcFinding 進行聚合、分組和統計 ● 協調並調度數據處理和 Plotly 繪圖 ● 最終的靜態網頁檔案

第三節、程式原始碼

```

import ast
import sys
import os
import javalang          # 需要安裝: pip install javalang
import pycparser          # 需要安裝: pip install pycparser
from pycparser import c_parser, c_ast, parse_file
import pandas as pd        # 數據處理
import plotly.graph_objects as go # 視覺化圖表
import json                # JSON 輸出
from datetime import datetime # 獲取時間戳
import webbrowser

# --- PQC 知識庫與修復建議 (PQC_KNOWLEDGE_BASE) ---
PQC_KNOWLEDGE_BASE = {
    # 弱雜湊 (Priority Fixes)
    "B303": {"type": "WEAK_HASH_SHA1", "message": "使用了 SHA1 雜湊算法。", "fix": "替換為 hashlib.sha256/sha3, SHA1 對碰撞攻擊是脆弱的。"},

```

```

    "B324": {"type": "WEAK_HASH_MD5", "message": "使用了 MD5 雜湊算法。",
    "fix": "必須移除 MD5，替換為 SHA256。"},
    # 弱加密算法 (Priority Fixes)
    "B304": {"type": "WEAK_CIPHER_DES", "message": "使用了 DES/3DES 弱加密算法。", "fix": "停用 DES/3DES，改用 AES-256 GCM 模式。"},
    # 量子脆弱資產與使用樣式 (PQC/AES)
    "B413_RSA": {"type": "PQC_TARGET_RSA", "message": "發現 RSA 密鑰生成。", "fix": "量子脆弱：考慮替換為 CRYSTALS-Kyber (KEM) 或 Dilithium (Signature)。"},
    "B413_AES_WEAK": {"type": "WEAK_CIPHER_MODE", "message": "使用了不安全的 AES/ECB 模式。", "fix": "替換為 AES-256 GCM 或 CCM 模式，確保認證性。"},
    "B413_AES_SAFE": {"type": "TRADITIONAL_AES_ASSET", "message": "使用了 AES 加密資產。", "fix": "這是一個抗量子資產。請確保 IV/Nonce 是正確生成的。"},
    "B413_RSA_WEAK_SIZE": {
        "type": "WEAK_ASSET_RSA",
        "message": "發現 RSA 密鑰長度小於 2048 bits，對暴力破解脆弱。",
        "fix": "將密鑰長度至少增加到 2048/4096 bits，並規劃 PQC 遷移。"},
    },
    # CBC/CFB 模式 IV 缺失 (使用樣式風險)
    "B413_IV_WEAK": {
        "type": "WEAK_IV_NONCE",
        "message": "在 CBC/CFB 模式中，未偵測到 IV/Nonce 參數，易受重放攻擊。",
        "fix": "必須使用 os.urandom (Python) 或 SecureRandom (Java) 創建隨機 IV。"},
    },
    # 量子脆弱的 ECC
    "B413_ECC": {
        "type": "PQC_TARGET_ECC",
        "message": "發現 ECC/ECDSA/ECDH 橢圓曲線加密資產。",
        "fix": "核心量子脆弱資產，建議替換為 CRYSTALS-Dilithium/Falcon。"},
    },
    # 硬編碼偵測
    "B105_HARDCODED_SECRET": {
        "type": "SECRET_LEAKAGE",
        "message": "發現硬編碼密鑰，可能導致密鑰洩露，影響 PQC 遷移後的安全性。",

```

```

    "fix": "將密鑰儲存於環境變數或專門的密鑰管理器中。"
},
# --- PQC 正面識別 (PQC Ready) ---
"B501_KYBER": {"type": "PQC_KEM_ML_KEM", "message": "發現 NIST 標準
PQC 算法：ML-KEM (Kyber)。", "fix": "PQC READY。請確保實作符合 FIPS 203 標
準。"},
"B502_DILITHIUM": {"type": "PQC_SIGN_ML_DSA", "message": "發現 NIST
標準 PQC 算法：ML-DSA (Dilithium)。", "fix": "PQC READY。請確保實作符合
FIPS 204 標準。"},
# --- [HARDCORE] 硬編碼與機密管理 ---
"B702_HARDCODED_KEY": {"type": "HARDCODED_SECRET_KEY", "message": "
偵測到疑似硬編碼的加密金鑰。", "fix": "絕對禁止在程式碼中寫死金鑰。請改用環境
變數或 KMS。"},
"B706_HARDCODED_PASSWORD": {"type": "HARDCODED_PASSWORD", "message":
"偵測到疑似硬編碼的密碼。", "fix": "請勿將密碼儲存在原始碼中。"},
"B707_HARDCODED_AWS": {"type": "HARDCODED_CLOUD_CREDENTIAL",
"message": "偵測到硬編碼 AWS Key (AKIA...)。", "fix": "使用 IAM Role。"},
"B708_HARDCODED_TOKEN": {"type": "HARDCODED_API_TOKEN", "message": "
偵測到疑似硬編碼 API Token。", "fix": "動態生成 Token。"},
"B709_HARDCODED_PQC_SK": {"type": "HARDCODED_PQC_PRIVATE_KEY",
"message": "偵測到疑似 PQC 私鑰硬編碼。", "fix": "PQC 私鑰極為敏感。"},
"B701_WEAK_RNG": {"type": "WEAK_RANDOM_SOURCE", "message": "使用弱亂
數 (random)。", "fix": "改用 os.urandom。"},

# --- [ADVANCE] 進階參數檢查 ---
"B415_ECC_WEAK_CURVE": {"type": "WEAK_ECC_CURVE", "message": "弱橢圓
曲線 (如 P-192)。", "fix": "使用 NIST P-256 以上。"},
"B703_WEAK_KDF_ITERATIONS": {"type": "WEAK_KDF_ITERATION_COUNT",
"message": "PBKDF2 迭代次數過低。", "fix": "建議 > 600,000 次。"},
"B710_SHORT_SALT": {"type": "INSUFFICIENT_SALT_LENGTH", "message":
"Salt 長度不足。", "fix": "Salt 應 > 16 bytes。"},
"B416_GCM_NONCE_LENGTH": {"type": "RISKY_GCM_NONCE_LENGTH",
"message": "GCM Nonce 非 12 bytes。", "fix": "固定為 12 bytes。"},
}
# -----

# --- 核心邏輯：報告生成 (作為獨立函數) ---
def report_finding(node, filename, line, rule_id, custom_message=None):

```

```

    info = PQC_KNOWLEDGE_BASE.get(rule_id, {"type": "UNKNOWN",
"message": "未知規則", "fix": "N/A"})

    # 根據節點類型獲取代碼片段（適應 Python, Java, C）
    if isinstance(node, str):
        code_snippet = node
    elif isinstance(node, (ast.Call, ast.Attribute)):
        code_snippet = ast.unparse(node).strip()
    elif hasattr(node, 'value'):
        # 適用於 javalang 的 Literal 節點
        code_snippet = str(node.value).strip('')
    elif hasattr(node, 'name'):
        # 適用於 C AST (FuncCall)
        code_snippet = str(node.name) if isinstance(node,
c_ast.FuncCall) else str(node)
    else:
        code_snippet = str(node)

    location_str = f"{filename}:{line}" if line > 0 else
f"{filename}:N/A"

    return {
        "RuleID": rule_id,
        "Type": info.get('type', 'UNKNOWN_TYPE'),
        "Location": location_str,
        "CodeSnippet": code_snippet,
        "Message": custom_message if custom_message else
info.get('message', 'N/A'),
        "FixSuggestion": info.get('fix', 'N/A')
    }

def _determine_pqc_status(rule_id):
    """決定資產的 PQC 狀態（用於 CBOM）"""
    if "HARDCODED" in rule_id: return "CRITICAL_SECRET_LEAK"
    if any(k in rule_id for k in ["SHA1", "MD5", "DES"]): return
"VULNERABLE (CLASSIC)"
    if any(k in rule_id for k in ["RSA", "ECC", "WEAK"]): return
"VULNERABLE (QUANTUM)"

```

```

        if any(k in rule_id for k in ["KYBER", "DILITHIUM"]): return
"PQC_READY"
        if "AES" in rule_id and "SAFE" in rule_id: return "SAFE (QUANTUM-
RESISTANT)"
        return "UNKNOWN"

# --- Python 掃描核心 ---
class PQC_AST_Visitor(ast.NodeVisitor):
    def __init__(self, filename, findings_list):
        self.filename = filename
        self.findings_list = findings_list

    def _get_literal_value(self, node):
        if isinstance(node, ast.Constant): return node.value
        return None

    def _get_call_arg_value(self, node, arg_index, kw_name):
        val = None
        for k in node.keywords:
            if k.arg == kw_name and isinstance(k.value, ast.Constant):
                val = k.value.value
        if val is None and len(node.args) > arg_index:
            if isinstance(node.args[arg_index], ast.Constant): val =
node.args[arg_index].value
        return val

    def visit_Assign(self, node):
        target_name = ""
        for target in node.targets:
            if isinstance(target, ast.Name):
                target_name = target.id.lower()
                break
        if not target_name:
            self.generic_visit(node)
            return

        # 獲取字面量值 (修正 UnboundLocalError)
        raw_value = self._get_literal_value(node.value)

```



```

        assigned_value = None
        if isinstance(raw_value, str): assigned_value = raw_value
        elif isinstance(raw_value, bytes):
            try: assigned_value = raw_value.decode('utf-8')
            except: assigned_value = str(raw_value)

        # 檢查邏輯
        if assigned_value and len(assigned_value) > 8:
            if assigned_value.startswith(("AKIA", "ASIA")):
                self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B707_HARDCODED_AWS"))
            elif any(s in target_name for s in ['password', 'passwd',
'pwd']) and "hash" not in target_name:
                self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B706_HARDCODED_PASSWORD"))
            elif ("token" in target_name or "api_key" in target_name)
and "csrf" not in target_name and len(assigned_value) > 10:
                self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B708_HARDCODED_TOKEN"))
            elif ("sk" in target_name or "secret_key" in target_name)
and ("pqc" in target_name or "kyber" in target_name):
                self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B709_HARDCODED_PQC_SK"))
            elif any(s in target_name for s in ['key', 'secret',
'private']):
                if "public" not in target_name and "pub" not in
target_name:
                    self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B702_HARDCODED_KEY"))
                self.generic_visit(node)# 確保繼續遍歷子節點

    def visit_Call(self, node):
        full_name = self._get_full_name(node.func)

        # 1. 弱雜湊 (最高優先級別)
        if "hashlib.sha1" in full_name:
            self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B303"))

```

```

        elif "hashlib.md5" in full_name:
            self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B324"))

        elif "random.random" in full_name or "random.randint" in
full_name:
            self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B701_WEAK_RNG"))

# 2. 量子脆弱/弱加密 (DES, RSA)
        elif any(x in full_name for x in ["DES.new", "DES3.new",
"Crypto.Cipher.DES"]):
            self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B304"))

        elif "RSA.generate" in full_name:
            key_size = self._get_int_arg(node.args, 0)
            if key_size is not None and key_size < 2048:
                self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B413_RSA_WEAK_SIZE"))
            else:
                self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B413_RSA"))

# 3. AES 模式檢查 (修正邏輯，確保 ECB/IV 缺失優先被檢查)
        elif "AES.new" in full_name:
            is_ecb = self._is_ecb_mode(node)
            is_cbc_cfb = self._is_cbc_cfb_mode(node)
            iv_is_missing = not self._has_keyword_arg(node.keywords,
'iv')

            # 確保最危險的模式優先被標記 (ECB)
            if is_ecb:
                finding = report_finding(node, self.filename,
node.lineno, "B413_AES_WEAK")

                # 其次檢查 IV 缺失 (使用樣式漏洞)
                elif is_cbc_cfb and iv_is_missing:
                    finding = report_finding(node, self.filename,
node.lineno, "B413_IV_WEAK")

```

```

        # 最後，如果通過所有漏洞檢查，則視為安全資產
        else:
            finding = report_finding(node, self.filename,
node.lineno, "B413_AES_SAFE")

            self.findings_list.append(finding)

    if node.args or node.keywords:
        args_str = ""
        try:
            # 將所有參數轉為字串以進行關鍵字搜索
            args_str = ", ".join([ast.unparse(a) for a in node.args])
            args_str += ", ".join([ast.unparse(k.value) for k in
node.keywords])
        except: pass

        args_str = args_str.upper()
        if "KYBER" in args_str or "ML-KEM" in args_str:
            self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B501_KYBER"))
        elif "DILITHIUM" in args_str or "ML-DSA" in args_str:
            self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B502_DILITHIUM"))

    if "PBKDF2" in full_name:
        iters = self._get_call_arg_value(node, 3, 'iterations')
        if iters is not None and isinstance(iters, int) and iters <
600000:
            self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B703_WEAK_KDF_ITERATIONS"))

    if "generate_private_key" in full_name and "ec" in full_name:
        for k in node.keywords:
            if k.arg == 'curve':
                val = ast.unparse(k.value).upper() if hasattr(ast,
'unparse') else ""
                if any(w in val for w in ['SECP192', 'SECT163',
'BRAINPOOLP160']):

```

```

        self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B415_ECC_WEAK_CURVE"))

    elif "ECC.generate" in full_name:
        is_weak_curve = False
        for k in node.keywords:
            # 檢查 curve='P-192' 等弱曲線
            if k.arg == 'curve':
                val = ast.unparse(k.value).upper() if hasattr(ast,
'unparse') else ""
                if any(w in val for w in ['P-192', 'SECP192',
'BRAINPOOLP160']):
                    self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B415_ECC_WEAK_CURVE"))
                    is_weak_curve = True

        # 如果不是弱曲線，它仍然是 PQC 遷移目標 (ECC 本身對量子脆弱)
        if not is_weak_curve:
            self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B413_ECC"))

    # --- [B710] Salt 長度檢查 (針對 os.urandom) ---
    # 檢查: os.urandom(N) 其中 N < 16
    if "os.urandom" in full_name:
        size = self._get_int_arg(node.args, 0)
        # 排除 12 (GCM Nonce 標準長度)，只針對過短的 Salt/IV
        if size is not None and size < 16 and size != 12:
            self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B710_SHORT_SALT"))

    # --- [B416] AES-GCM Nonce 長度檢查 ---
    # 檢查: AES.new(..., nonce=os.urandom(N)) 其中 N != 12
    if "AES.new" in full_name:
        # 檢查是否使用了 GCM 模式
        is_gcm = False
        for k in node.keywords:
            if k.arg == 'mode' and 'GCM' in
ast.unparse(k.value).upper():

```

```

        is_gcm = True
        break

    # 如果是 GCM，檢查 nonce 參數
    if is_gcm:
        for k in node.keywords:
            if k.arg == 'nonce':
                # 檢查 nonce 是否來自 os.urandom
                if isinstance(k.value, ast.Call) and "urandom" in
ast.unparse(k.value.func):
                    nonce_size = self._get_int_arg(k.value.args,
0)
                    if nonce_size is not None and nonce_size !=
12:
                        self.findings_list.append(report_finding
(node, self.filename, node.lineno, "B416_GCM_NONCE_LENGTH"))

    # 確保繼續遍歷子節點
    self.generic_visit(node)

# 辅助函数：获取完整函数名
def _get_full_name(self, node):
    if isinstance(node, ast.Attribute):
        return self._get_full_name(node.value) + "." + node.attr
    elif isinstance(node, ast.Name):
        return node.id
    return ""

# 辅助函数：检查 ECB 模式
def _is_ecb_mode(self, call_node):
    for keyword in call_node.keywords:
        if keyword.arg == 'mode':
            return 'ECB' in ast.unparse(keyword.value).upper()
    # 检查第二個位置參數
    if len(call_node.args) > 1:
        return 'ECB' in ast.unparse(call_node.args[1]).upper()
    return False

```

```

# 辅助函数: 检查 CBC/CFB 模式 (需要 IV)
def _is_cbc_cfb_mode(self, call_node):
    for keyword in call_node.keywords:
        if keyword.arg == 'mode':
            mode = ast.unparse(keyword.value).upper()
            return 'CBC' in mode or 'CFB' in mode
    # 检查第二个位置参数
    if len(call_node.args) > 1:
        mode = ast.unparse(call_node.args[1]).upper()
        return 'CBC' in mode or 'CFB' in mode
    return False

# 辅助函数: 检查关键字参数是否存在
def _has_keyword_arg(self, keywords, arg_name):
    return any(keyword.arg == arg_name for keyword in keywords)

# 辅助函数: 获取整数参数 (Key Size)
def _get_int_arg(self, args, index):
    if len(args) > index:
        arg = args[index]
        if isinstance(arg, ast.Constant) and isinstance(arg.value,
int):
            return arg.value
    return None

def visit_Constant(self, node):
    """
    捕捉所有字符串常数, 用于识别 PQC 关键字 (Kyber, Dilithium)
    适用于 Python 3.8+ (旧版 Python 使用 visit_Str)
    """
    if isinstance(node.value, str):
        val = node.value.upper()
        # 检查 PQC 关键字
        if "KYBER" in val or "ML-KEM" in val:
            self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B501_KYBER"))
        elif "DILITHIUM" in val or "ML-DSA" in val:

```

```

        self.findings_list.append(report_finding(node,
self.filename, node.lineno, "B502_DILITHIUM"))

        # 繼續遍歷 (雖然 Constant 通常是葉節點)
        self.generic_visit(node)

def scan_python(filepath):
    findings_list = []
    with open(filepath, 'r', encoding='utf-8') as f:
        code = f.read()
    tree = ast.parse(code, filename=filepath)
    visitor = PQC_AST_Visitor(filepath, findings_list)
    visitor.visit(tree)
    return findings_list

# --- Java 掃描核心 ---

def is_secret_var(name):
    """ 判斷變數名稱是否敏感 """
    sensitive = ['key', 'secret', 'password', 'passwd', 'pwd', 'token',
'private', 'credential']
    name = name.lower()
    return any(k in name for k in sensitive) and "public" not in name
and "hash" not in name

def scan_java(filepath):
    findings_list = []
    with open(filepath, 'r', encoding='utf-8') as f:
        code = f.read()

    try:
        # javalang 解析器
        tree = javalang.parse.parse(code)

    except javalang.tokenizer.LexerError as e:
        # 捕獲詞法錯誤 (例如非法字符)，返回錯誤資訊
        print(f"✗ Java Lexer Error (可能為非法字符或 BOM): {e}")
        return []

```

```

except javalang.parser.ParserError as e:
    # 捕獲語法錯誤 (例如缺少分號或類別名錯誤)
    print(f"✗ Java Parser Error (語法錯誤或結構不完整): {e}")
    return []
except Exception as e:
    # 捕獲其他所有錯誤
    print(f"✗ Java AST 錯誤: {e}")
    return []

# --- 成功解析後，開始遍歷 AST ---
for path, node in tree:
    try:
        # 確保 node 是一個 javalang AST 節點， path 是節點路徑
        if not isinstance(node, javalang.tree.Node):
            continue
    except ValueError:
        # 捕獲 too many values to unpack 錯誤
        # 這表示 javalang 返回的不是 (path, node) 格式
        continue

    line_num = node.position.line if node.position else 0

    # 如果行號為 0 (缺失)，嘗試從路徑中回溯到最近的父節點
    if line_num == 0:
        # 關鍵修正：將 path 迭代包裹在 try-except 塊中，以防 path 內部結構不穩定
        try:
            for p_item in reversed(path):
                # p_item 應該是 (attribute_name, p_node)
                if len(p_item) == 2:
                    p_node = p_item[1]
                    if p_node.position:
                        line_num = p_node.position.line
                        break
        except Exception:
            # 捕獲 path 迭代時的解包錯誤
            pass

```



```

# 排除掉頂層的 PackageDeclaration 或 Import 語句
if line_num == 0 and isinstance(node,
(javalang.tree.PackageDeclaration, javalang.tree.Import)):
    continue

# 1. 方法呼叫檢查 (MethodInvocation)
if isinstance(node, javalang.tree.MethodInvocation):

    # [getInstance 檢查]
    if node.member == 'getInstance':
        if node.arguments and isinstance(node.arguments[0],
javalang.tree.Literal):
            arg_value =
node.arguments[0].value.strip('').upper()

            # === 構造清晰的代碼片段 (getInstance) ===
            qualifier = node.qualifier if node.qualifier else
"Cipher/Digest"
            readable_snippet =
f"{qualifier}.getInstance(\"{node.arguments[0].value.strip('\\')}\\")"

            # 規則匹配 (使用 readable_snippet)
            if "SHA1" in arg_value or "SHA-1" in arg_value:
                findings_list.append(report_finding(readable_snipp
et, filepath, line_num, "B303"))
            elif "MD5" in arg_value:
                findings_list.append(report_finding(readable_snipp
et, filepath, line_num, "B324"))
            elif "DES" in arg_value or "DESEDE" in arg_value:
                findings_list.append(report_finding(readable_snipp
et, filepath, line_num, "B304"))
            elif "AES" in arg_value:
                if "ECB" in arg_value:
                    findings_list.append(report_finding(readable_s
nippet, filepath, line_num, "B413_AES_WEAK"))
            else:

```

```

        findings_list.append(report_finding(readable_s
nippet, filepath, line_num, "B413_AES_SAFE"))
        elif "RSA" in arg_value:
            findings_list.append(report_finding(readable_snipp
et, filepath, line_num, "B413_RSA"))
        elif "EC" in arg_value or "ECDSA" in arg_value or
"ECDH" in arg_value:
            findings_list.append(report_finding(readable_snipp
et, filepath, line_num, "B413_ECC"))

# [initialize 檢查]
elif node.member == 'initialize':
    if len(node.arguments) == 1 and
isinstance(node.arguments[0], javalang.tree.Literal):
        try:
            key_size = int(node.arguments[0].value)
            readable_snippet =
f"keyPairGenerator.initialize({key_size})"

            if key_size < 2048:
                findings_list.append(report_finding(readable_s
nippet, filepath, line_num, "B413_RSA_WEAK_SIZE", f"RSA 金鑰過短
({key_size})"))
            else:
                findings_list.append(report_finding(readable_s
nippet, filepath, line_num, "B413_RSA", "RSA 金鑰生成 (PQC 目標)"))
        except ValueError:
            pass

# [弱亂數 nextInt/nextBytes 檢查]
elif node.member == 'nextInt' or node.member == 'nextBytes':
    if hasattr(node, 'qualifier') and node.qualifier and
'rand' in node.qualifier.lower() and 'secure' not in
node.qualifier.lower():
        readable_snippet =
f"{node.qualifier}.{node.member}(...)"
        findings_list.append(report_finding(readable_snippet,
filepath, line_num, "B701_WEAK_RNG"))

```

```

# 2. 變數宣告檢查 (LocalVariableDeclaration) - 硬編碼機密
elif isinstance(node, javalang.tree.LocalVariableDeclaration):
    for declarator in node.declarators:
        var_name = declarator.name.lower()

        # [硬編碼機密檢查]
        if declarator.initializer and
isinstance(declarator.initializer, javalang.tree.Literal):
            raw_value = str(declarator.initializer.value)

            if raw_value.startswith(''):
                value = raw_value.strip('')
                # === 構造硬編碼密鑰片段 ===
                readable_snippet = f"{declarator.name} =
\"{value[:15]}...\""

                if value.startswith("AKIA") or
value.startswith("ASIA"):
                    findings_list.append(report_finding(readable_s
nippet, filepath, line_num, "B707_HARDCODED_AWS"))
                elif is_secret_var(var_name):
                    if "password" in var_name:
                        findings_list.append(report_finding(readab
le_snippet, filepath, line_num, "B706_HARDCODED_PASSWORD"))
                    elif "token" in var_name:
                        findings_list.append(report_finding(readab
le_snippet, filepath, line_num, "B708_HARDCODED_TOKEN"))
                    elif "pqc" in var_name or "kyber" in var_name:
                        findings_list.append(report_finding(readab
le_snippet, filepath, line_num, "B709_HARDCODED_PQC_SK"))
                    else:
                        findings_list.append(report_finding(readab
le_snippet, filepath, line_num, "B702_HARDCODED_KEY"))

            # [Salt 長度檢查]
            if 'salt' in var_name and declarator.initializer:
                init = declarator.initializer

```

```

        readable_snippet = f"byte[] {declarator.name} = new
byte[...]"

        salt_size = None

        if isinstance(init, javalang.tree.ArrayCreator) and
init.dimensions and init.dimensions[0].value.isdigit():
            salt_size = int(init.dimensions[0].value)
        elif isinstance(init,
javalang.tree.ArrayInitializer):
            if init.initializers: salt_size =
len(init.initializers)

            if salt_size is not None and salt_size < 16:
                findings_list.append(report_finding(readable_snipp
et, filepath, line_num, "B710_SHORT_SALT"))

# 3. 類別創建檢查 (ClassCreator)
elif isinstance(node, javalang.tree.ClassCreator):
    type_name = node.type.name

    # [弱亂數]
    if type_name == 'Random':
        readable_snippet = "new Random()"
        findings_list.append(report_finding(readable_snippet,
filepath, line_num, "B701_WEAK_RNG"))

    # [PBKDF2 迭代次數]
    elif "PBEKeySpec" in type_name and len(node.arguments) >= 3:
        iter_arg = node.arguments[2]
        if isinstance(iter_arg, javalang.tree.Literal) and
iter_arg.value.isdigit():
            iterations = int(iter_arg.value)
            if iterations < 600000:
                readable_snippet = f"new {type_name}(...,
{iterations}, ...)"
                findings_list.append(report_finding(readable_snipp
et, filepath, line_num, "B703_WEAK_KDF_ITERATIONS"))

```

```

        # [ECC 曲線檢查]
        elif "ECGenParameterSpec" in type_name and
len(node.arguments) > 0:
            curve_arg = node.arguments[0]
            if isinstance(curve_arg, javalang.tree.Literal):
                curve_name = curve_arg.value.strip('').upper()
                readable_snippet = f"new
{type_name}(\\"{curve_name}\\")"

                if any(w in curve_name for w in ['SECP192',
'SECT163', 'BRAINPOOLP160']):
                    findings_list.append(report_finding(readable_snippet, filepath, line_num, "B415_ECC_WEAK_CURVE"))

        # [GCM Nonce 長度檢查]
        elif "GCMParameterSpec" in type_name and len(node.arguments)
>= 2:
            iv_arg = node.arguments[1]
            readable_snippet = f"new GCMParameterSpec(...)"
            if isinstance(iv_arg, javalang.tree.ArrayCreator):
                for dim in iv_arg.dimensions:
                    if isinstance(dim, javalang.tree.Literal) and
dim.value.isdigit():
                        size = int(dim.value)
                        if size != 12:
                            findings_list.append(report_finding(readable_snippet, filepath, line_num, "B416_GCM_NONCE_LENGTH"))

        # 4. 字串常數檢查 (PQC 識別)
        elif isinstance(node, javalang.tree.Literal):
            val = str(node.value)
            if val.startswith(''):
                val_clean = val.strip('').upper()
                if "KYBER" in val_clean or "ML-KEM" in val_clean or
"DILITHIUM" in val_clean or "ML-DSA" in val_clean:
                    readable_snippet = f\\"{val_clean}\\\"
                    if "KYBER" in val_clean or "ML-KEM" in val_clean:

```

```

        findings_list.append(report_finding(readable_snippet, filepath, line_num, "B501_KYBER"))
    elif "DILITHIUM" in val_clean or "ML-DSA" in val_clean:
        findings_list.append(report_finding(readable_snippet, filepath, line_num, "B502_DILITHIUM"))

    return findings_list

# --- C/C++ 掃描核心 ---
def scan_c_cpp(filepath):
    print(f"C/C++ 掃描邏輯尚未實作。")
    return []

# --- 主控函數 ---
def scan_project_recursive(root_dir):
    all_findings = []
    SUPPORTED_EXTENSIONS = ('.py', '.java', '.c', '.cpp')

    for dirpath, dirnames, filenames in os.walk(root_dir):
        if 'pqc_venv' in dirpath or '.git' in dirpath: # 忽略虚拟环境和 Git 目录
            continue

        for filename in filenames:
            if filename.endswith(SUPPORTED_EXTENSIONS):
                filepath = os.path.join(dirpath, filename)
                print(f"掃描檔案: {filepath}")

                try:
                    findings = scan_file(filepath)
                    all_findings.extend(findings)
                except Exception as e:
                    print(f"✗ 檔案 {filepath} 掃描失敗: {e}")

    return all_findings

```

```

def scan_file(filepath):
    if filepath.endswith(".py"):
        return scan_python(filepath)
    elif filepath.endswith(".java"):
        return scan_java(filepath)
    elif filepath.endswith(".c") or filepath.endswith(".cpp"):
        return scan_c_cpp(filepath)
    else:
        return []

# -----
# CBOM 視覺化和報告生成模塊
# -----

def generate_cbom_json(findings):
    """
    將掃描結果轉換為簡化的 CBOM (Cryptographic Bill of Materials) 格式。
    """
    cbom_data = {
        "metadata": {
            "tool": "PQC Hybrid Auditor",
            "version": "1.0",
            "total_findings": len(findings),
            "timestamp": datetime.now().isoformat()
        },
        "cryptographic_assets": []
    }

    for finding in findings:
        # 根據 RuleID 判斷資產類型 (簡化)
        asset_type = "ASYMMETRIC_PQC" if 'RSA' in finding['RuleID'] or
        'ECC' in finding['RuleID'] else "SYMMETRIC_HASH_ETC"

        cbom_data['cryptographic_assets'].append({
            "asset_id": finding['RuleID'],
            "location": finding['Location'],
            "type": asset_type,
            "code_snippet": finding['CodeSnippet'],

```

```

        "risk_status": finding['Type'],
    })

    return cbom_data

def generate_risk_pie_chart(findings):
    """
    使用 Plotly 生成風險分佈圓餅圖 (Pie Chart)，並輸出 HTML 字符串。
    """

    if not findings:
        return "<h3>未發現加密資產或弱點。</h3>"

    df = pd.DataFrame(findings)
    #risk_counts = df['Type'].value_counts()

    # 定義顏色：確保高風險 (WEAK, SECRET) 使用紅色/橙色
    color_map = {
        'WEAK_HASH_SHA1': '#D35400',      # 🔴 深焦橙 (高風險)
        'WEAK_HASH_MD5': '#C0392B',      # 🔴 深磚紅 (Critical)
        'WEAK_CIPHER_DES': '#C0392B',    # 🔴 深磚紅 (Critical)
        'WEAK_ASSET_RSA': '#D35400',      # 🔴 深焦橙 (高風險)
        'WEAK_CIPHER_MODE': '#C0392B',   # 🔴 深磚紅 (Critical)
        'WEAK_IV_NONCE': '#D35400',      # 🔴 深焦橙 (高風險)
        'PQC_TARGET_RSA': '#2980B9',     # 🔵 深海藍 (PQC 核心目標)
        'PQC_TARGET_ECC': '#2980B9',     # 🔵 深海藍 (PQC 核心目標)
        'TRADITIONAL_AES_ASSET': '#27AE60', # 🟢 翡翠綠 (安全資產)
        'SECRET_LEAKAGE': '#C0392B',      # 🔴 深磚紅 (Critical)
        'PQC_KEM_ML_KEM': '#2980B9',     # 🔵 深海藍 (PQC 核心目標)
        'PQC_SIGN_ML_DSA': '#2980B9',    # 🔵 深海藍 (PQC 核心目標)
        'HARDCODED_SECRET_KEY': '#C0392B', # 🔴 深磚紅 (Critical)
        'HARDCODED_PASSWORD': '#C0392B', # 🔴 深磚紅 (Critical)
        'HARDCODED_CLOUD_CREDENTIAL': '#C0392B', # 🔴 深磚紅 (Critical)
        'HARDCODED_API_TOKEN': '#C0392B', # 🔴 深磚紅 (Critical)
        'HARDCODED_PQC_PRIVATE_KEY': '#C0392B', # 🔴 深磚紅 (Critical)
        'WEAK_RANDOM_SOURCE': '#C0392B',  # 🔴 深磚紅 (Critical)
        'WEAK_ECC_CURVE': '#D35400',      # 🔴 深焦橙 (高風險)
        'WEAK_KDF_ITERATION_COUNT': '#D35400', # 🔴 深焦橙 (高風險)
        'INSUFFICIENT_SALT_LENGTH': '#D35400', # 🔴 深焦橙 (高風險)
    }

```



```

        'RISKY_GCM_NONCE_LENGTH': '#D35400',          # 🍊 深焦橙（高風險）
    }

    #colors = [color_map.get(label, '#95A5A6') for label in
risk_counts.index]

# 2. 統計數量並轉換為 DataFrame
stats = df['Type'].value_counts().reset_index()
stats.columns = ['Type', 'Count']

# 3. 對應顏色
# map 函式會根據 Type 填入對應的 Hex 色碼
stats['Color'] = stats['Type'].map(color_map).fillna('#95A5A6') # 預
設灰色

# 4. [關鍵步驟] 依照「顏色」進行排序
# 這樣相同的顏色（Hex Code）就會排在一起
# 第二排序鍵是 Count（降序），讓同顏色的區塊中，數量多的排前面
stats = stats.sort_values(by=['Color', 'Count'], ascending=[True,
False])

fig = go.Figure(data=[go.Pie(
    labels=stats['Type'],
    values=stats['Count'],
    hole=.4, # 甜甜圈圖
    marker=dict(colors=stats['Color']),
    hovertemplate='%{label}<br>數量: %{value}<extra></extra>' ,
    sort=False # [關鍵] 禁用 Plotly 的自動排序，強制使用我們上面排好的順
序
))]

fig.update_layout(
    title_text="PQC 遷移與弱點風險分佈（總資產數：
{}).format(len(findings)),
    title_x=0.5,
    font_color="#E0E0E0",          # 標題和圖例文字顏色（淺色）
    plot_bgcolor='#1E1E1E',        # 圖表繪圖區背景（深色）
    paper_bgcolor='#1E1E1E'        # 整個圖表紙張背景（深色）

```

```

)

# 转换为 HTML 字符串
return fig.to_html(full_html=False, include_plotlyjs='cdn')

def format_findings_table(findings):
    """将详细发现列表格式化为 HTML 表格"""
    if not findings:
        return ""

    html = '<table border="1" style="width:100%; border-collapse: collapse;">'
    html += '<tr style="background-color:#eee;"><th>#</th><th>位置</th><th>類型</th><th>代碼片段</th><th>修補建議</th></tr>'

    for i, f in enumerate(findings):
        color = 'red' if 'WEAK' in f['Type'] or 'SECRET' in f['Type']
        else 'blue'

        # 避免代碼片段破坏 HTML 结构
        code_safe = f['CodeSnippet'].replace('<', '&lt;').replace('>', '&gt;').replace('\n', '<br>').replace('|', '/')

        html += f"""
        <tr>
            <td>{i+1}</td>
            <td>{f['Location']}</td>
            <td><strong style="color: {color};">{f['Type']}</strong></td>
            <td><code>{code_safe}</code></td>
            <td>{f['FixSuggestion']}</td>
        </tr>
        """

    html += '</table>'
    return html

def generate_full_report_html(findings):
    """生成包含圖表和表格的最終 HTML 報告"""

```

```

chart_html = generate_risk_pie_chart(findings)
table_html = format_findings_table(findings)

# 設置深色模式的顏色代碼
BG_COLOR = '#121212' # 極深灰色/接近黑色
TEXT_COLOR = '#e0e0e0' # 淺灰色 (適合深色模式的文字顏色)
CONTAINER_BG = '#1e1e1e' # 內容框的深灰色

final_html = f"""
<!DOCTYPE html>
<html lang="zh-TW">
<head>
    <meta charset="UTF-8">
    <title>PQC 混合審計儀表板</title>
    <script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
    <style>
        body {{
            font-family: Arial, sans-serif;
            margin: 20px;
            background-color: {BG_COLOR};
            color: {TEXT_COLOR}; /* 全局文字顏色 */
        }}
        .container {{
            max-width: 1200px;
            margin: auto;
            background: {CONTAINER_BG}; /* 內容框背景 */
            padding: 20px;
            box-shadow: 0 0 10px rgba(0,0,0,0.5);
            border-radius: 8px;
        }}
        h1 {{ color: {TEXT_COLOR}; }} /* 標題顏色 */
        /* 修正表格背景，確保在深色背景下可讀 */
        table tr th {{ background-color: #333; color: #fff; }}
        table tr td {{ border-color: #444; }}
    </style>
</head>

```

```

<body>
    <div class="container">
        <h1>PQC 混合審計工具報告</h1>
        <p><strong>掃描時間:</strong>
{pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S')}</p>
        <p><strong>總發現資產與弱點數:</strong> {len(findings)}</p>

        <h2>風險分佈儀表板</h2>
        <div id="plotly-chart">
            {chart_html}
        </div>

        <h2>詳細資產與漏洞清單</h2>
        {table_html}
    </div>
</body>
</html>
"""

return final_html

# --- 主程序入口修改（調用報告生成函數） ---

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("用法: python3 pqc_ast_scanner.py <檔案路徑> | <目錄路徑>")
        sys.exit(1)

    path_to_scan = sys.argv[1]

    if not os.path.exists(path_to_scan):
        print(f"❌ 錯誤: 找不到路徑 {path_to_scan}。")
        sys.exit(1)

    try:
        if os.path.isdir(path_to_scan):
            findings = scan_project_recursive(path_to_scan)
        else:
            findings = scan_file(path_to_scan)

```

```

except Exception as e:
    print(f"致命錯誤：掃描過程中發生異常：{e}")
    sys.exit(1)

# 1. 生成 CBOM JSON
cbom_json_content = generate_cbom_json(findings)
cbom_filename = "PQC_CBOM_Inventory.json"
with open(cbom_filename, 'w', encoding='utf-8') as f:
    json.dump(cbom_json_content, f, indent=4)

# 2. 生成 HTML 視覺化報告
HTML_FILENAME = "PQC_Risk_Dashboard.html"
full_html_content = generate_full_report_html(findings)

try:
    with open(HTML_FILENAME, 'w', encoding='utf-8') as f:
        f.write(full_html_content)

    # 取得 HTML 檔案的絕對路徑（確保瀏覽器能正確找到檔案）
    file_path = os.path.abspath(HTML_FILENAME)

    # 3. 輸出到終端機（簡化輸出）
    print("\n" + "=" * 60)
    print("☑ 掃描完成！")
    print(f"總發現問題數：{len(findings)}")
    print(f"    -> 加密資產清單：{cbom_filename}")
    print(f"    -> 視覺化報告：{HTML_FILENAME}（請在瀏覽器中打開此文件查看儀表板）")
    print("=" * 60)

    # 3. [新增] 自動開啟瀏覽器
    print(f"🔗 正在開啟瀏覽器檢視報告...")
    webbrowser.open(f"file://{file_path}")

except Exception as e:
    print(f"✗ 寫入報告失敗：{e}")

```

第五章、系統測試

第一節、目的說明

本章節旨在說明本專題的測試策略、執行過程、以及結果分析。

核心目的：透過整合測試和系統測試兩階段，驗證系統是否符合《系統需求規格書》（SRS）中定義的功能需求（FR）與非功能需求（NFR）。

重點驗證：確保跨語言 AST 解析的準確性、PQC 規則匹配的精準度，以及最終 CBOM 與 HTML 報告輸出的穩定性與標準性。

第二節、測試環境

一、硬體配備

項目	規格	備註
處理器（CPU）	Intel Core i5	用於確保 NFR-P1（掃描效率）的基礎測試。
記憶體（RAM）	16 GB	確保在掃描大型專案時，能滿足 NFR-P2（資源消耗）的測試條件。
儲存設備	1024 GB SSD	

二、軟體配備與架構說明

項目	版本	說明
作業系統	Windows 11 / macOS	測試系統的跨平台兼容性。
核心語言	Python 3.9+	運行核心掃描器。
核心依賴	javalang, pycparser, pandas, plotly	確保所有第三方函式庫的集成正常運作。
架構	CLI 驅動的單體架構	系統以命令列模式運行，所有功能皆封裝在單一 Python 應用程式中。

第三節、測試計畫

一、整合測試計畫（Integration Testing）

驗證獨立開發的模組（如 Scanner、Parser、RuleEngine、Reporter）之間協作與數據傳輸的正確性。

1. 項目

編號	項目名稱	目的
IT-01	Scanner & Parser 整合	驗證 ProjectScanner 能正確分派檔案，並確保 PythonParser 和 JavaParser 能成功將原始碼轉換為 AST 結構。
IT-02	AST & RuleEngine 整合	驗證各語言的 Visitor 能將 AST 節點資訊正確傳遞給 RuleEngine，並確保規則引擎能依據 PQC 知識庫準確生成 PqcFinding 物件。
IT-03	Engine & Reporter 整合	驗證 RuleEngine 產生的 PqcFinding 列表能被 Reporter 模組接收，並成功轉換為 CBOM JSON 和 Plotly 數據。

2. 方法與工具

- 方法：採用自頂向下（Top-Down）的整合方式，從 CLI 啟動開始，逐步驗證核心模組的流程。
- 工具：手動中斷點：在數據傳輸的關鍵節點（例如 RuleEngine 接收數據前）設置中斷點，手動檢查 PqcFinding 物件的內容和結構是否符合預期。
- 測試數據：使用小型、結構明確的測試檔案（僅包含 1-2 個已知弱點），確保流程的單一性。

3. 結果

項目編號	測試結果	記錄與說明
IT-01	通過	成功遍歷多層級目錄，並將 .py 和 .java 檔案交給對應的解析器，AST 樹結構生成成功。
IT-02	通過	規則引擎在接收 AST 節點後，能根據 KBase 規則，精確識別弱點並生成標準化的 PqcFinding 數據結構。
IT-03	通過	PqcFinding 列表成功被 pandas 結構化，並正確轉換為 CBOM JSON 檔案和 Plotly 繪圖所需的數據格式。

二、系統測試計畫（System Testing）

模擬真實環境運行，驗證整個系統是否滿足 SRS 中定義的所有功能和功能性

要求。

1. 項目

編號	測試項目名稱	追溯需求
ST-01	功能完整性測試	FR-01, FR-02, FR-04
ST-02	CBOM 報告與標準性測試	FR-05
ST-03	HTML 視覺化與可用性測試	FR-06, NFR-U3

2. 方法與工具

方法：採用黑箱測試策略，僅關注 CLI 輸入和報告輸出，驗證系統的最終結果。

工具：

- Gemini 輔助生成數據：請 Gemini 根據 PQC 規則（如 RSA 弱密鑰、ECB 模式）生成正向和反向的 Python/Java 測試代碼，專門用於驗證規則的準確性（ST-01）。
- GitHub 開源測試數據：導入在 GitHub 上找到的 Python 和 Java 開源弱點專案，用於測試真實世界的複雜度和規模（ST-01, ST-04）。

3. 結果

項目編號	測試結果	記錄與說明
ST-01	通過	使用 Gemini 生成的已知弱點測試集進行測試，成功識別所有目標弱點。
ST-02	通過	輸出的 CBOM JSON 經結構驗證工具檢查，確認符合 CycloneDX Schema，並成功記錄 pqc_status: VULNERABLE (QUANTUM) 屬性。
ST-03	通過	HTML 報告能穩定在 Chrome/Edge 瀏覽器中開啟，Plotly 圖表可懸停顯示數據，符合 NFR-U3。

三、測試結果示意圖

下圖展示同時掃描 .py 和 .java 檔的結果：

```
(venv) PS C:\Users\Caesar\Desktop\Project-PQC> python pqc_ast_scanner.py C:\Users\Caesar\Desktop\Project-PQC\Gemini_test
[+] 掃描完成！
總發現問題數：47
-> 加密資產清單：PQC_CBOM_Inventory.json
-> 視覺化報告：PQC_Risk_Dashboard.html (請在瀏覽器中打開此文件查看儀表板)
=====
🔗 正在開啟瀏覽器檢視報告...
```

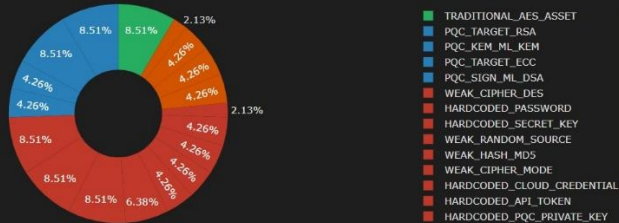

PQC 混合審計工具報告

掃描時間: 2025-12-17 00:37:43

總發現資產與弱點數: 47

風險分佈儀表板

PQC 遷移與弱點風險分佈 (總資產數: 47)



20	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:131	HARDCODED_PQC_PRIVATE_KEY	pqcPrivateKey = "-----BEGIN PQC ..."	PQC 私鑰極為敏感。
21	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:131	PQC_KEM_ML_KEM	"-----BEGIN PQC PRIVATE KEY-----...KYBERSK..."	PQC READY。請確保實作符合 FIPS 203 標準。
22	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:N/A	WEAK_RANDOM_SOURCE	new Random()	改用 os.urandom。
23	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:140	WEAK_RANDOM_SOURCE	weakRand.nextInt(...)	改用 os.urandom。
24	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:144	HARDCODED_PASSWORD	password = "password..."	請勿將密碼儲存在原始碼中。
25	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:151	INSUFFICIENT_SALT_LENGTH	byte[] shortSalt = new byte[...]	Salt 應 > 16 bytes。
26	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\pqc_test_suite.py:23	WEAK_HASH_SHA1	hashlib.sha1(data)	替換為 hashlib.sha256/sha3，SHA1 對碰撞攻擊是脆弱的。
27	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\pqc_test_suite.py:27	WEAK_HASH_MD5	hashlib.md5(data)	必須移除 MD5，替換為 SHA256。
28	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\pqc_test_suite.py:39	WEAK_CIPHER_DES	DES.new(key_des, DES.MODE_ECB)	停用 DES/3DES，改用 AES-256 GCM 模式。
29	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\pqc_test_suite.py:40	WEAK_CIPHER_DES	DES3.new(b'16bytekey16bytek', DES3.MODE_ECB)	停用 DES/3DES，改用 AES-256 GCM 模式。
30	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\pqc_test_suite.py:48	HARDCODED_SECRET_KEY	<ast.Constant object at 0x000002AC105C169b>	絕對禁止在程式碼中寫死金鑰。請改用環境變數或 KMS。
31	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\pqc_test_suite.py:49	WEAK_CIPHER_MODE	AES.new(key, AES.MODE_ECB)	替換為 AES-256 GCM 或 CCM 模式，確保認證性。
32	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\pqc_test_suite.py:55	TRADITIONAL_AES_ASSET	AES.new(key, AES.MODE_GCM, nonce=bad_nonce)	這是一個抗量子資產。請確保 IV/Nonce 是正確生成的。
33	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\pqc_test_suite.py:60	TRADITIONAL_AES_ASSET	AES.new(key, AES.MODE_CBC, iv=static_iv)	這是一個抗量子資產。請確保 IV/Nonce 是正確生成的。
34	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\pqc_test_suite.py:68	WEAK_ASSET_RSA	RSA.generate(1024)	將密鑰長度至少增加到 2048/4096 bits，並規劃 PQC 遷移。

#	位置	類型	代碼片段	修復建議
1	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:28	WEAK_HASH_MD5	MessageDigest.getInstance("MD5")	必須移除 MD5，替換為 SHA256。
2	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:32	WEAK_HASH_SHA1	MessageDigest.getInstance("SHA-1")	替換為 hashlib.sha256/sha3，SHA1 對碰撞攻擊是脆弱的。
3	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:41	WEAK_CIPHER_DES	Cipher.getInstance("DES/ECB/PKCS5Padding")	停用 DES/3DES，改用 AES-256 GCM 模式。
4	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:44	WEAK_CIPHER_DES	Cipher.getInstance("DESede/ECB/PKCS5Padding")	停用 DES/3DES，改用 AES-256 GCM 模式。
5	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:55	WEAK_CIPHER_MODE	Cipher.getInstance("AES/ECB/PKCS5Padding")	替換為 AES-256 GCM 或 CCM 模式，確保認證性。
6	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:59	TRADITIONAL_AES_ASSET	Cipher.getInstance("AES/GCM/NoPadding")	這是一個抗量子資產。請確保 IV/Nonce 是正確生成的。
7	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:69	TRADITIONAL_AES_ASSET	Cipher.getInstance("AES/CBC/PKCS5Padding")	這是一個抗量子資產。請確保 IV/Nonce 是正確生成的。
8	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:80	PQC_TARGET_RSA	KeyPairGenerator.getInstance("RSA")	量子脆弱：考慮替換為 CRYSTALS-Kyber (KEM) 或 Dilithium (Signature)。
9	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:81	WEAK_ASSET_RSA	keyPairGenerator.initialize(1024)	將密鑰長度至少增加到 2048/4096 bits，並規劃 PQC 遷移。
10	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:84	PQC_TARGET_RSA	KeyPairGenerator.getInstance("RSA")	量子脆弱：考慮替換為 CRYSTALS-Kyber (KEM) 或 Dilithium (Signature)。
11	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:85	PQC_TARGET_RSA	keyPairGenerator.initialize(2048)	量子脆弱：考慮替換為 CRYSTALS-Kyber (KEM) 或 Dilithium (Signature)。
12	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:88	PQC_TARGET_ECC	KeyPairGenerator.getInstance("EC")	核心量子脆弱資產，建議替換為 CRYSTALS-Dilithium/Falcon。
13	C:\Users\Caesar\Desktop\Project-PQC\Gemini_test\PQCTestSuite.java:N/A	WEAK_ECC_CURVE	new ECGenParameterSpec("SECP192K1")	使用 NIST P-256 以上。

第六章、使用手冊

第一節、目的說明

提供使用者一個清晰、逐步的操作指南，使其能夠成功地安裝、運行並維護本專題的靜態分析工具。

內容：摘要描述本系統的必要軟硬體環境、完整的安裝部署步驟、CLI 命令列操作方法，以及後續的系統維護程序。

目標：確保使用者能夠高效地利用本工具進行專案的後量子密碼學（PQC）脆弱性盤點與報告生成。

第二節、系統需求

描述系統運作所需的軟體和硬體配置，以確保掃描器能夠穩定且高效地運行。

一、硬體需求

項目	建議最低配置	說明
處理器 (CPU)	雙核心 (Dual-Core) 或更高	用於處理 AST 遞迴解析和規則匹配的計算密集型任務。
記憶體 (RAM)	8 GB	用於應對大規模專案掃描時，暫存 AST 結構和 Pandas 數據處理所需。
儲存空間	1 GB 可用空間	專案程式碼、Python 環境、函式庫和報告輸出檔案所需。

二、軟體需求

項目	建議版本	說明
作業系統	Windows 10/11, macOS, Linux (Ubuntu/CentOS)	系統需在主流作業系統上具備跨平台運行能力。
程式語言環境	Python 3.9 +	運行核心掃描器和所有依賴函式庫所需。
瀏覽器	Chrome, Edge, Firefox	用於查看 Plotly 生成的 HTML 網頁報告 (FR-06)。

第三節、安裝程序

描述安裝系統所需的步驟，將系統部署到使用者環境中。

步驟一：準備環境與程式碼

1. 安裝 Python：確保系統中已安裝 Python 3.8+。
2. 克隆專案：從 GitHub 倉庫下載專案程式碼。

指令碼：

```
git clone https://github.com/amberyul227/PQC-AST-Scanner.git
cd PQC-AST-Scanner
```

3. 建立虛擬環境：

指令碼：

```
python -m venv pqc_env
source pqc_env/bin/activate # Linux / macOS
.\pqc_env\Scripts\activate # Windows
```

步驟二：安裝依賴函式庫

在虛擬環境啟動後，使用 pip 安裝所有必要的第三方函式庫。

指令碼：

```
pip install javalang pycparser pandas plotly cyclonedx-python-lib
```

步驟三：驗證安裝

運行簡單的 CLI 幫助指令，確認系統可以正確啟動。

指令碼：

```
python pqc_ast_scanner.py --help
```

第四節、操作程序

描述系統的主要操作步驟和使用方法，透過命令列介面（CLI）啟動掃描。

一、系統操作步驟

步驟	動作	說明
1	執行掃描指令	在終端機中，使用 <code>python [主程式名稱]</code> 並指定 <code>--path</code> 參數。
2	指定目標路徑	<code>--path</code> 參數需指定要掃描的專案根目錄或單一檔案路徑。
3	等待結果	系統將自動執行 AST 解析、規則匹配，並在控制台顯示進度。

4	檢視報告	掃描完成後，生成的報告檔案將存放在執行目錄中。
---	------	-------------------------

二、核心 CLI 使用方法

CLI 至少支援以下格式：

```
python pqc_ast_scanner.py --path <project-dir> --output <report-prefix>
```

參數	格式範例	說明
--path	./my_project	【必填】 指定要進行 PQC 盤點的專案目錄或檔案路徑。
--output	results	【選填】 設定輸出報告的檔案名稱前綴。將生成 results_CBOM.json 和 results_Report.html。

三、報告檢視

1. CBOM 報告：使用任何文本編輯器或 JSON 檢視器打開 [output]_CBOM.json 檔案。
2. HTML 報告：直接在瀏覽器中雙擊打開 [output]_Report.html 檔案，以檢視 Plotly 互動式圖表。

第五節、維護程序

描述系統的維護方法和注意事項，以確保系統長期運行的穩定性與規則的時效性。

一、系統維護方法

維護項目	執行方法	說明
規則庫更新 (FR-03)	僅需修改 PQC 知識庫部分	這是最常見的維護工作。隨著新的 PQC 算法和遷移指南發布，需新增或修改規則。
函式庫更新	運行 <code>pip install --upgrade [library_name]</code>	定期更新 javalang, pycparser, pandas, plotly 等依賴庫，以確保解析器和報告功能的穩定性。
性能優化	程式碼重整	針對 NFR-P1 性能指標，定期檢查 AST 遍歷器和規則引擎的邏輯，消除性能瓶頸。

二、注意事項

1. 程式碼格式：由於 AST 解析器對程式碼的語法要求嚴格，請確保輸入的原始碼沒有明顯的語法錯誤，以避免解析器崩潰。
2. 數據備份：建議定期備份生成的 CBOM JSON 檔案，作為 PQC 遷移進度的歷史紀錄。

第七章、進度報告

第一節、目的說明

1. 報告進度：參照專題計畫中的時程規劃，向老師報告專題製作的進展。
2. 諮詢製作專題所遭遇的困難與問題：關於時程的延宕、意見的分歧與所遭遇的困難，與老師進行討論尋求解答。
3. 會議規劃：一個月一次與指導老師進行專題進度報告會議。

第二節、進度表

一、第一次會議記錄

1. 時間：10/17
2. 地點：線上會議
3. 參與人員：黃馨賢、李昺園、余采芸、陳威棋 副總、Chole 老師、TonTon 老師
4. 本次進度完成項目：決定好之後專題發展的方向。
5. 下次進度欲完成項目：使用一些開源工具，比較各工具之間的優缺點。
6. 諮詢問題與結果：
Q: 如何實作後量子密碼遷移之加密演算法自動化盤點？
A: 透過 github 使用開源工具，例如 PQCA/sonar-cryptography 這個 snoarqube 的外掛。

二、第二次會議紀錄

1. 時間：11/17
2. 地點：線上會議
3. 參與人員：黃馨賢、李昺園、余采芸、陳威棋 副總、Chole 老師、TonTon 老師
4. 本次進度完成項目：
 - 使用了 PQCA/sonar-cryptography 這個 SonarQube 外掛、semgrep、bandit。
 - 完成一個小 Demo，並且查找相關 PQC 規則加入進實作的程式碼內，開發針對不同程式語言的掃描核心。
5. 下次進度欲完成項目：
 - 完善程式碼內容。
 - 決定要結合先有掃描工具的擷取與整合並新增規則，或者開發一個新的工具和規則庫。

第八章、專題結論

第一節、專題成果心得

在專題初期，因為對於後量子密碼遷移完全不了解，在實務中也是一個很新的概念，所以前期花了很多時間在閱讀文獻資料。接下來，我們實際安裝並操作了幾個靜態程式掃描工具，像是 Sonarqube（裝 PQCA plugin）、Bandit 和 Semgrep，並大概了解它們的運作邏輯，做了一個可以統整這三個工具輸出報告的小 Demo。在決定專題的目標、方向及規劃也是碰壁了很久，要從現有的多個開源靜態程式掃描工具的基礎上，擷取關於後量子密碼的部分再整合？還是自己開發一個工具並加入規則？最後我們選擇後者，因為這些開源的掃描工具每個都是用不同的語言寫的，而且寫規則的邏輯也完全不同，因此難以整合。

在實作的初期，本來想利用 tree-sitter 來進行靜態分析，但在安裝環境的時候遇到了問題，因此改成使用 AST 來進程式碼的掃描，除了能掃描也要有判定的標準，哪些演算法符合哪些不符合，是否需要更改。一開始我們只是單純的透過抓取關鍵字來做判定，但後面陸續加入了其他的規則，而且因為我們做的是多語言的掃描，除了 python 的掃描核心要做還要做 java 的，我覺得這是整個專題中最重要且困難的部分。後來也改進了掃描完後輸出的資料，本來都是直接顯示在終端機中，但這樣每行看下來會很雜亂、難以分辨，所以後面就做了視覺化的報告出來，利用圓餅圖的樣式搭配顏色可以很簡單的就明白當前程式碼的安全程度。

第二節、未來展望

一、 智能化與修復增強

利用人工智慧技術，將工具的能力從「發現問題」提升到「解決問題」。

領域	規劃項目	目標
大語言模型整合 (LLM)	結合 LLM API。	語義級修復建議：不僅指出錯誤行號，而是根據程式碼的上下文和專案的語言版本，動態生成精確的修復程式碼片段，大幅減少開發者手動修改的時間。
誤報過濾優化	導入機器學習分類器。	降低誤報率：訓練模型識別常見的良性代碼模式（如註解中的密鑰名稱、測試代碼中的弱算法）。

二、 服務化與深度整合

將 CLI 工具轉變為企業級的持續監控服務，實現自動化運行。

領域	規劃項目	目標
----	------	----

Web 應用介面 (Streamlit)	利用 Streamlit 建立 Web 儀表板和互動式掃描介面。	用戶體驗升級：允許使用者透過瀏覽器上傳專案文件或輸入 Git 倉庫 URL 觸發掃描。同時，將 Plotly HTML 報告直接嵌入 Streamlit 儀表板中，提供即時且集中化的風險視圖。
CI/CD 服務化	將掃描器 Docker 容器化，並整合至 CI/CD 流程。	自動化品管閘門：在程式碼提交、合併請求時自動觸發掃描，將 PQC 檢查作為程式碼品管的強制閘門，阻止有風險的代碼進入主分支。

三、 語言與規則擴展

增加工具的適用範圍和分析的深度。

領域	規劃項目	目標
規則庫國際化	擴展 PQC 知識庫的規則種類。	NIST 標準覆蓋：納入更多 NIST PQC 標準化算法的用法規則，例如 Dilithium 和 Kyber 等算法的最佳實踐和潛在錯誤用法。
配置文件分析	增加對 YAML, XML, JSON 配置文件的解析器。	間接資產識別：分析專案的 pom.xml (Maven)、build.gradle (Gradle) 或 requirements.txt (Python) 等文件，識別和報告依賴項中使用的密碼學庫版本和潛在風險。
程式語言擴展	實現對 C/C++ 的 AST 解析。	市場覆蓋率提升：儘管架構中已預留 scan_c_cpp 函數，但解析邏輯尚未實作。

第三節、銘謝

衷心感謝來自勤業眾信的指導老師們。在專題初期，面對後量子密碼學這一個陌生的概念，老師們提供了專題訂定方向與可行性目標，還有後量子密碼的相關論文及工具，幫助我們從眾多開源工具的繁雜基礎中，確立了自主開發基於 AST 靜態分析工具的核心方向。在這學期的專題製作中，老師們透過每月定期的線上會議，持續關心並了解我們的進度。對於我們在實作過程中，不論是技術方面，還是目標確立方面，老師們總能耐心給予專業的解答與指導，使我們克服困難、將專題從單純的輸出報告整合，進化到運用精確的 AST 語義分析和視覺化報告，成功自己做出一個符合後量子密碼遷移的靜態掃描盤點工具。

參考文件

NIST PQC 標準化 (NIST PQC Standardization)

<https://csrc.nist.gov/Projects/post-quantum-cryptography>

OWASP Top 10 - A02:2021 (Cryptographic Failures)

https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

CycloneDX CBOM 規格書 (CycloneDX Specification)

<https://cyclonedx.org/specification/overview/>

Bandit (Python Security Scanner)

<https://bandit.readthedocs.io/en/latest/>

Semgrep (Static Analysis)

<https://semgrep.dev/docs/>

Python ast 模組官方文件

<https://docs.python.org/3/library/ast.html>

javalang 函式庫 (GitHub)

<https://github.com/c2nes/javalang>

pycparser 函式庫 (GitHub)

<https://github.com/eliben/pycparser>

Pandas (數據分析)

<https://pandas.pydata.org/docs/>

Plotly Python (互動式繪圖)

<https://plotly.com/python/>

java 測試資料 (GitHub)

<https://github.com/bcgit/bc-java>

python 測試資料 (GitHub)

<https://github.com/pyca/cryptography/tree/main/tests>